

Redes Neuronales Recurrentes

Modelando dependencias temporales y secuencias complejas (RNN, LSTM, GRU)

Jan Polanco Velasco

¿Por qué el orden importa?

El problema de la "Bolsa de Palabras" (*Bag of Words*)

- Consideremos la siguiente oración incompleta:

"El gato se comió el _____"

- ¿Cómo sabe un modelo si sigue "pescado" y no "avión"?
- **Las redes densas (MLP) tradicionales:**
 - Tratan las entradas como un saco desordenado de características (*i.i.d.*).
 - Pierden completamente la sintaxis, el contexto y la estructura temporal

Mismo conjunto de palabras, significado opuesto:

1. "No estaba mal, estaba **excelente**." → **Positivo (+1)**

2. "No estaba excelente, estaba **mal**." → **Negativo (-1)**

Una red densa cuenta las palabras idénticas y arroja la misma predicción.
Una **RNN** procesa palabra por palabra manteniendo un vector de memoria interna.

La Neurona Recurrente Básica

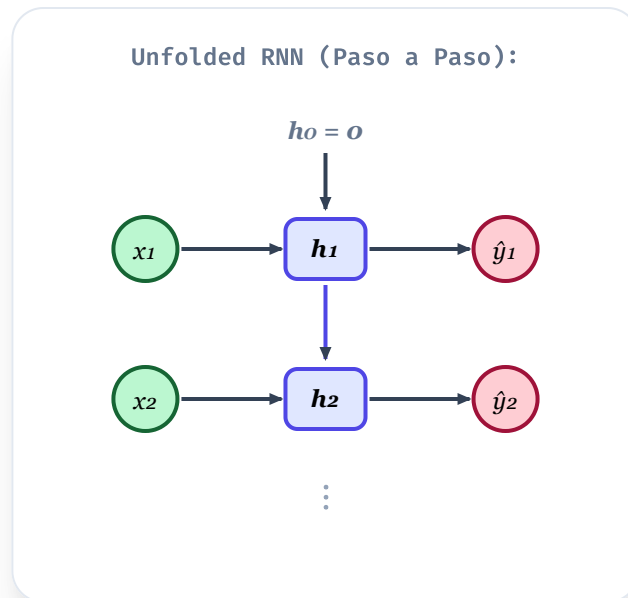
Desenrollado Temporal (*Unfolding through time*)

- **Vector de Estado Oculto (h_t):** Representa la memoria acumulada hasta el instante t .
- **Condición inicial:** En el instante cero $h_0 = \mathbf{0}$ (sin recuerdos previos).
- **Ecuaciones fundamentales en cada paso t :**

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$$

$$\hat{y}_t = g(W_{hy}h_t + b_y)$$

- **Pesos compartidos:** Las matrices W_{xh} , W_{hh} , W_{hy} son **las mismas** en cada paso del tiempo.



Patrones de Uso en Secuencias

Taxonomía de Arquitecturas Recurrentes

- **Many-to-One:** Secuencia de entrada → Una sola predicción al final (*Análisis de sentimientos, clasificación de audios*).
- **One-to-Many:** Una sola entrada semilla → Secuencia completa de salida (*Generación de música, subtulado de imágenes*).
- **Many-to-Many Sincronizado** ($T_x = T_y$): Salida por cada instante de entrada (*POS Tagging gramatical, predicción de series*).
- **Many-to-Many Asíncrono** ($T_x \neq T_y$): Modelo Encoder-Decoder (*Traducción de idiomas*,



El Talón de Aquiles de la RNN Simple

Desvanecimiento del Gradiente (*Vanishing Gradient*)

- **Backpropagation Through Time (BPTT):** Para ajustar los pesos, el gradiente debe retroceder multiplicándose en cada paso temporal:

$$\frac{\partial \mathcal{L}_T}{\partial h_1} = \frac{\partial \mathcal{L}_T}{\partial h_T} \prod_{j=2}^T \frac{\partial h_j}{\partial h_{j-1}}$$

- **El producto sucesivo de derivadas:**
 - Si los eigenvalores de $W_{hh} < 1$ o la derivada de $\tanh < 1$, el gradiente se atenúa exponencialmente hacia **cero**.

- **Consecuencia práctica:**
 - La red sufre de **amnesia a corto plazo**: no recuerda palabras o datos vistos hace más de 8 o 10 pasos atrás.

⚠ **Pérdida de Dependencias a Largo Plazo**

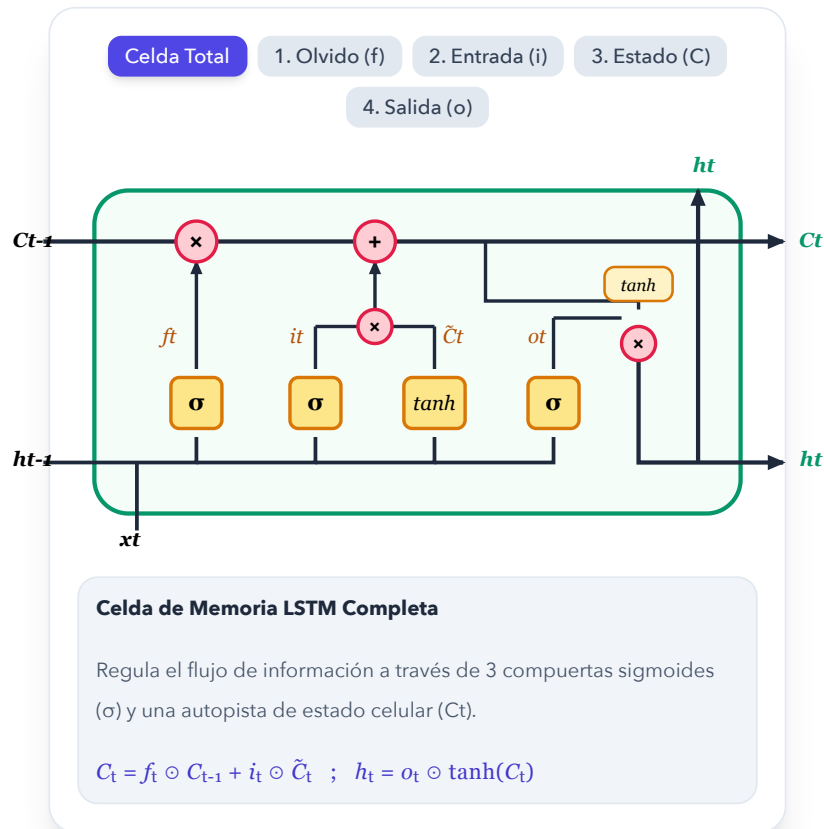
"Los **perros** que vi corriendo por el parque detrás de los niños durante aquella fría tarde de invierno en Bogotá estaban / estaba cansados."

Para cuando la RNN llega al verbo final, la influencia del gradiente del sujeto plural ("Los perros") se ha desvanecido por completo.

LSTM (Long Short-Term Memory)

Hochreiter & Schmidhuber (1997)

- **La Celda de Memoria:** Diseñada explícitamente para mantener información a través de cientos de pasos.
- **La Autopista de Estado Celular (C_t):**
 - Flujo de información lineal con muy pocas transformaciones no lineales.
 - El gradiente fluye hacia atrás mediante sumas sin desvanecerse.
- **Las 3 Compuertas Moduladoras (σ):**
 1. **Compuerta de Olvido (f_t):** Qué descartar del pasado.
 2. **Compuerta de Entrada (i_t):** Qué nuevo conocimiento registrar.
 3. **Compuerta de Salida (o_t):** Qué enviar al estado oculto actual h_t .



GRU: Gated Recurrent Unit

Cho et al. (2014) – La alternativa simplificada

- **Motivación:** LSTM es potente pero costosa en cómputo (4 matrices de pesos por celda).
- **Fusión de estados:** Unifica el estado celular C_t y el estado oculto h_t en una sola variable: h_t .
- **Solo 2 compuertas principales:**
 - Γ_u (**Update Gate**): Decide si conservar el estado anterior o actualizarlo.
 - Γ_r (**Reset Gate**): Decide qué tanto del estado anterior influye en el nuevo candidato $\tilde{h}_t$.
- **Actualización convexa elegante:**

LSTM vs. GRU

Eficiencia

LSTM

3 Compuertas (f, i, o)
2 Estados (c, h)

$4 \times d^2$ Parámetros

GRU

2 Compuertas (r, u)
1 Estado (h)

$3 \times d^2$ Parámetros (25% menos)

GRU converge significativamente más rápido en conjuntos de datos medianos o con recursos limitados.

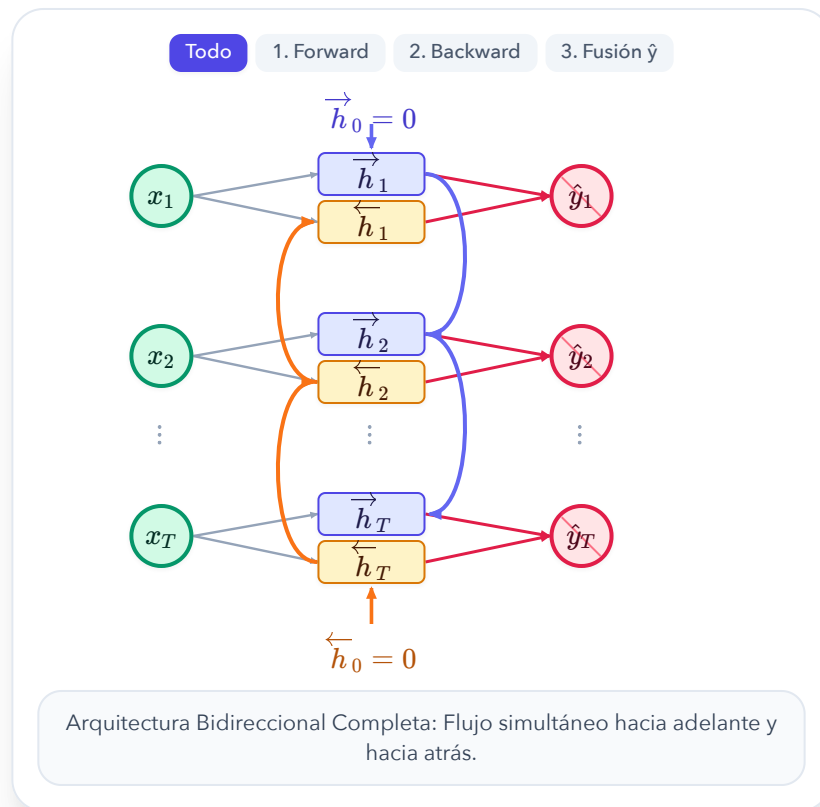
Redes Recurrentes Bidireccionales

Pasado, Presente y Futuro simultáneos

- **Limitación unidireccional:** Una RNN tradicional en el paso t solo conoce información anterior ($x_1, \dots, x_t$).
- **Solución Bidireccional (BiRNN / BiLSTM):**
 - Capa hacia adelante ($\vec{h}_t$): Lee la secuencia de izquierda a derecha.
 - Capa hacia atrás ($\overleftarrow{h}_t$): Lee la secuencia de derecha a izquierda.
- **Salida combinada:**

$$\hat{y}_t = g \left(W_y [\vec{h}_t, \overleftarrow{h}_t] + b_y \right)$$

- **Requisito fundamental:** Toda la secuencia debe estar disponible con antelación (*no aplicable a predicción financiera en tiempo real*).



¿Siguen siendo relevantes las LSTM frente a Transformers?

Desafíos de LSTM

- **Naturaleza estrictamente secuencial:** No se pueden paralelizar en GPU durante el entrenamiento.
- **Degradación en secuencias extensas:** En secuencias de más de 500 pasos, el cuello de botella vectorial olvida detalles finos.

Complejidad temporal: $O(T)$

¿Dónde siguen ganando?

- **Dispositivos embebidos y Edge AI:** Consumen una fracción insignificante de memoria RAM frente a un Transformer.
- **Inferencia paso a paso en tiempo real:** Streaming de audio o sensores industriales de bajo costo.
- **Bajo volumen de datos:** No requieren millones de parámetros pre-entrenados para converger.

Huella de memoria: Mínima

La Era Transformer

El mecanismo de **Atención (Self-Attention)** permite conectar cualquier par de palabras directamente en $O(1)$ operaciones sin depender de un estado recurrente.

Base de GPT, Llama y BERT



Laboratorio Práctico de la Sesión 2

De la teoría a la práctica en Python con TensorFlow / Keras

1

Modelado de Series Temporales con LSTM

Manejo del tensor tridimensional [samples, time_steps, features]

2

Benchmarking: Simple RNN vs. LSTM vs. GRU

Comparación de velocidad de convergencia, loss y número de parámetros

3

Inferencia Recurrente y Proyecciones Futuras

Evaluación de estabilidad y retención de memoria a largo plazo